AutoVet Testing Guide

What each test is, how to run it yourself in PowerShell, and how to explain it to the panel.

#	Test	Tool	Expected result
0	Setup: run `start-test-server.cmd`	One script	It prints READY
1	Unit testing: backend + frontend	PHPUnit, Vitest	14 passed, 11 passed
2	Integration testing: AI module	Postman / Newman	8 assertions, 0 failed
3	System testing: patient journey	Postman / Newman	15 assertions, 0 failed
4a	Security testing: role-based access	Postman / Newman	17 assertions, 0 failed
4b	Load testing: 30 users at once	Node.js script	Both thresholds ✓, 0% errors
–	Troubleshooting		

0. Setup

First time only: install the test tools.

```
cd C:\laragon\www\AutoVet
npm install
cd C:\laragon\www\AutoVet\frontend\admin
npm install
```

This installs Newman (Postman's command-line runner) and Vitest into the project, so `npx newman` and `npx vitest` run without asking to download anything.

Every practice session: run the start script.

```
C:\laragon\www\AutoVet\docs\postman\testing\start-test-server.cmd
```

It does everything for you and checks each part:

1. Starts **MySQL** if it is not running.
2. Opens a new window called **"AutoVet TEST SERVER - keep this window open"**. This is the AutoVet API on `http://127.0.0.1:8010`, using the test copy of the database.
3. Checks that the API answers.

Wait until it prints **READY**. Then run every test in **your** PowerShell window (the one where you ran the script), **not** in the TEST SERVER window.

> **Do not close the "AutoVet TEST SERVER" window** while testing. If it is closed, every Postman test fails with `ECONNREFUSED 127.0.0.1:8010`. Just run the script again to bring it back. Close that window when you are done practicing.

> The test database `autovet_qa` already exists. It is a **copy** of your local database, so tests can add and change data without touching your real data or production. Every account in it uses the password `password123`.

> All commands in this guide are for **Windows PowerShell**. Copy them exactly. In PowerShell, environment settings are written as `$env:NAME = "value"`.

1. Unit Testing

> **In simple terms:** test one small piece at a time, like checking each part of a car before assembling it. Example: "if a dog is 3 months old, is it labeled Baby?"

What we test

- **Backend (PHPUnit):** pet age group, the low-stock alert, VAT calculation (12% already included in prices), and the forecast accuracy formulas.
- **Frontend (Vitest + React Testing Library):** the appointment calendar displays correctly, and the login form shows the right messages.

How to run: backend

```
cd C:\laragon\www\AutoVet\backend
php artisan test tests/Feature/BusinessRulesUnitTest.php tests/Unit/ForecastMetricsTest.php
```

Expected: 14 passed (45 assertions). These tests use their own database (`autovet_test`), which is wiped and rebuilt on every run. The first test takes about a minute because of that rebuild.

How to run: frontend

```
cd C:\laragon\www\AutoVet\frontend\admin
npx vitest run --reporter=verbose
```

Expected: 11 passed. No server or database is needed for these tests.

> **Say to the panel:** *We wrote PHPUnit tests for the business rules in Laravel. We create a pet with a known birth date and check the age group, sell stock down to the reorder level and check that the low-stock alert fires, and check that the VAT inside ₱1,120 is ₱120. For the frontend, Vitest and React Testing Library render the calendar and the login form in a test browser environment and check what the user would see.*

Files to show if asked

- `backend/tests/Feature/BusinessRulesUnitTest.php`
- `frontend/admin/src/test/` (calendar and login tests)

About the test server (what the start script sets up)

Tests 2, 3 and 4a send real requests to the AutoVet API, so the API must be running. The start script runs it with these settings, only inside the TEST SERVER window:

Setting	What it does
`DB_DATABASE=autovet_qa`	Uses the test copy of the database, not your real data.
`AI_API_URL=null`	The forecast runs on the Python model on your PC, the same way production does.
`MAIL_MAILER=log`, `BROADCAST_CONNECTION=log`	Emails and real-time messages are only written to the log, never sent.
`QUEUE_CONNECTION=sync`	Runs background jobs immediately, like the live server.
`--no-reload`	Required on Windows. Without it, the Python forecast fails (WinError 10106).

To check the server at any time:

```
Test-NetConnection 127.0.0.1 -Port 8010 | Select-Object TcpTestSucceeded
```

TcpTestSucceeded : True means it is running. **False** means run the start script again.

Using the Postman app instead of the terminal (optional)

1. In Postman, click **Import** and select everything in `C:\laragon\www\AutoVet\docs\postman\testing\` ending in `.json`.
2. In the top-right environment menu, choose **AutoVet QA (local)**.
3. Open a collection, click **Run collection**, then **Run**. Postman shows each test as passed or failed.

The terminal command (Newman) and the Postman app run exactly the same tests. Newman is simply Postman's command-line runner.

2. Integration Testing (AI module)

> **In simple terms:** check that separate parts work **together**. Laravel reads the usage history from MySQL, hands it to Python, Python predicts, and the answer comes back to the dashboard as JSON.

How to run

After the start script prints READY, in the same PowerShell window:

```
cd C:\laragon\www\AutoVet\docs\postman\testing
npx newman run 01-Integration-AI-Forecast.postman_collection.json -e AutoVet-QA.postman_environment.json
```

Expected: 8 assertions, 0 failed.

What happens, step by step

1. Log in as the clinic admin.
2. Read the item's current stock straight from the database.
3. Call the forecast. Postman checks that the answer came from the Python scikit-learn model, used the same stock that is in MySQL, learned a positive usage rate, was validated on held-out data (80/20 split), and predicts a future stock-out date.

> **Say to the panel:** *Postman first reads the item's stock straight from the database, then calls the forecast endpoint. We check that the answer really came from the Python scikit-learn model, that it used the same stock that is in MySQL, and that it was validated on held-out data with an 80/20 split. That proves the modules talk to each other correctly.*

3. System Testing (patient journey)

> **In simple terms:** test the **whole** system like a real day at the clinic, from start to finish, with each person using their own account.

The 9 steps

#	Step	Who
1	Logs in to the portal	Client
2	Books an appointment online (status: pending)	Client
3	Logs in	Clinic admin
4	Approves the booking	Clinic admin
5	Logs in	Veterinarian
6	Adds the medical record	Veterinarian
7	Reads the vaccine stock before billing	Clinic admin
8	Creates the finalized invoice	Clinic admin
9	Checks the stock went down by exactly 1	Clinic admin

How to run

```
cd C:\laragon\www\AutoVet\docs\postman\testing
npx newman run 02-System-Patient-Journey.postman_collection.json -e AutoVet-QA.postman_environment.json
```

Expected: 15 assertions, 0 failed. Each run creates a new appointment and invoice, and the vaccine stock goes down by 1. That is normal.

> **Say to the panel:** *We ran the full patient journey through the real API, with each step done by the correct role: the client books, the admin approves, the vet records the visit, the invoice is finalized, and Postman confirms the vaccine stock dropped by exactly one without any manual adjustment.*

4a. Security Testing (who can do what)

> **In simple terms:** try to do things you are **not allowed** to do, and confirm the system blocks you. In this test, getting **"403 Forbidden"** counts as a pass.

What it checks

Who	Tries to	Result
No login	Read appointments	Blocked (401)
Client	Change clinic settings, view staff accounts, read audit logs, book for someone else's pet	Blocked (403)
Client	Open their own portal	Allowed (200)
Veterinarian	Create a medical record	Allowed (201)
Staff	View that medical record	Allowed (200)
Staff	Delete it, create one, change settings	Blocked (403)
Clinic admin	Open the super-admin area	Blocked (403)
Clinic admin	Check the record still exists after staff tried to delete it	Still there (200)

How to run

```
cd C:\laragon\www\AutoVet\docs\postman\testing
npx newman run 03-Security-RBAC.postman_collection.json -e AutoVet-QA.postman_environment.json
```

Expected: 17 assertions, 0 failed.

> **Say to the panel:** *We logged in as each role and tried actions that role should not be able to do. Laravel blocks them with role middleware and policies. For example, the MedicalRecordPolicy only lets clinical roles delete records, so staff get 403. We also confirmed the record was still there afterwards.*

4b. Load Testing (30 users at once)

> **In simple terms:** have 30 staff accounts use the system at the same time and measure whether it stays fast. The target is the dashboard loading in **under 3 seconds**.

The live server runs **4 PHP workers with opcache** (a speed setting), so the test copies that setup. The opcache settings are already in the project (`docs\postman\testing\php-ini\zz-opcache.ini` , the same values as production). The TEST SERVER window (port 8010) is not used here, but run the start script first anyway: it makes sure MySQL is running.

> **Before you run it:** the server, MySQL and all 30 virtual users run on your one laptop, so the result depends on how fast the laptop is at that moment.
>
> - **Plug in the charger.** On battery, Windows slows the CPU down and the test fails (we measured dashboards taking 7 seconds on battery, versus 0.16 seconds plugged in).
> - Set Windows power mode to **Best performance** (Settings → System → Power & battery).
> - Close heavy apps you don't need (extra Chrome tabs, Excel).
> - Make sure the live-database sync (`db:sync-live`) is not running; it uses the same MySQL.

Step 1: start 4 workers

Open **4 PowerShell windows**. Paste this into each one, changing only the port on the last line: **8011**, **8012**, **8013**, **8014**. Leave all 4 windows open.

```
cd C:\laragon\www\AutoVet\backend
$env:PHP_INI_SCAN_DIR = "C:\laragon\www\AutoVet\docs\postman\testing\php-ini"
$env:DB_DATABASE = "autovet_qa"
$env:AI_API_URL = "null"
$env:MAIL_MAILER = "log"
$env:BROADCAST_CONNECTION = "log"
php artisan serve --port 8011 --no-reload
```

Step 2: check all 4 workers are running (in a 5th window)

```
8011..8014 | ForEach-Object { "$_ : " + (Test-NetConnection 127.0.0.1 -Port $_).TcpTestSucceeded }
```

All four lines must end in **True**.

Step 3: run the load test (in the same 5th window)

```
cd C:\laragon\www\AutoVet\docs\postman\testing
$env:BASE_URLS = (8011..8014 | ForEach-Object { "http://127.0.0.1:$_" }) -join ","
$env:ACCOUNTS = @(
  "admin@autovet.com",
  "maria.santos@petwellness.ph",
  "ramon.delacruz@petwellness.ph",
  "aileen.reyes@petwellness.ph",
  "joy.mendoza@petwellness.ph",
  "carlo.bautista@petwellness.ph",
  "scan.clinic_admin@autovet.local",
  "scan.veterinarian@autovet.local",
  "scan.staff@autovet.local"
) -join ","
node load-test.js 30 30 3000
```

The first line builds the 4 worker addresses (ports 8011 to 8014). The second lists the 9 staff accounts the 30 virtual users log in with.

The numbers mean **30** users, for **30** seconds, with a **3000** ms (3 second) target.

Expected: both thresholds show ✓, 0.00% errors, and the dashboard p(95) in the hundreds of milliseconds. Your exact numbers will differ a little on every run. That is normal.

How to read the result

Line	Meaning
`dashboard_load_time avg`	Average time to fully load the dashboard
`p(95)`	95% of dashboard loads were faster than this
`http_req_failed`	Percentage of requests that failed (should be 0.00%)
`http_reqs`	Total requests sent, and requests per second

> **Say to the panel:** *We ran 30 virtual users at once for 30 seconds, each logged in with a real staff account. Each one opens the dashboard, which makes 6 API calls in parallel like the browser does, opens another page, then pauses 4 to 6 seconds like a real person reading. We measured 1,267 requests with 0% errors. The dashboard loaded in 160 ms on average, and 95% of loads finished under 278 ms, far below our 3-second target.*

If they ask

- **"Why not k6?"** We used a Node.js script built for our dashboard's real request pattern. It measures the same things k6 does: response time percentiles and error rate.
- **"Was this on the live server?"** No. It ran on a local copy with the same worker count and opcache settings as production, so we would not overload the live free-tier server. Live response times would be slower, because that server has much less CPU.

Troubleshooting

Problem	Fix
PHPUnit: `SQLSTATE[HY000] [2002] No connection could be made because the target machine actively refused it`	MySQL is not running. Run `start-test-server.cmd` (it starts MySQL), wait for READY, then run the tests again.
Newman: `connect ECONNREFUSED 127.0.0.1:8010` and `JSONError "undefined" is not valid JSON`	The test server is not running, or its window was closed. Run `start-test-server.cmd`, wait for READY, then run Newman again.
The start script prints `FAILED`	Read the message. If it is about MySQL, open Laragon and click Start All, then run the script again. If it is about the server, read the error in the "AutoVet TEST SERVER" window.
Newman asks `Need to install the following packages: newman ... Ok to proceed?`	The test tools are not installed yet. Do the first-time `npm install` in Setup.
AI test fails: `expected 'Insufficient Data' to deeply equal 'Success'`	The server was started some other way, without `--no-reload`. Close that server window and use `start-test-server.cmd`.
Patient journey: `422` on booking, "This pet already has 2 appointment(s) booked for this day"	The clinic's rule of at most 2 bookings per pet per day. The test picks a random date each run, so just run it again.
Load test shows `ECONNREFUSED 127.0.0.1:8011` (or 8012–8014)	One of the 4 worker windows is not running.
Load test: `✗ 'p(95)<3000ms'` with dashboards taking several seconds, but `http_req_failed` is 0.00%	The laptop is too slow at that moment, usually because it is **on battery**. Plug in the charger, set power mode to Best performance, close heavy apps, wait for `db:sync-live` to finish, then run it again.
Load test: `login failed for ...: 401`	The 4 workers are not using the test database. Check each worker window has the `$env:DB_DATABASE = "autovet_qa"` line, with the quotes, then restart them.
Login fails with `429`	You ran the collections more than about 10 times in one minute. Wait 1 minute and try again.
`Failed to listen on 127.0.0.1:8011` (port in use)	An old worker window is still open on that port. Close it and start again.
The first PHPUnit test takes about a minute	Normal: the test database is rebuilt before the tests run.
Forecast test is slow (a few seconds)	Normal: it starts Python and trains the model on each call.

Where everything is

What	Location

Backend unit tests	`backend\tests\Feature\BusinessRulesUnitTest.php`, `backend\tests\Unit\ForecastMetricsTest.php`
Frontend tests	`frontend\admin\src\test\`
Start script (MySQL + test server)	`docs\postman\testing\start-test-server.cmd`
Postman test collections + environment	`docs\postman\testing\`
Load test script and opcache settings	`docs\postman\testing\load-test.js`, `docs\postman\testing\php-ini\`
Results with screenshots	`docs\AutoVet-Testing-Activities.pdf`